

seven segment display

```
#include <avr/io.h>
#include <util/delay.h>
```

```
uint8_t digits[10] = { 0x3f, 0x06, 0x5b, 0x4f,
                       0x66, 0x6d, 0x7d, 0x07, 0x7f,
                       0x6f };
```

```
int main (void) {
    DDAD = 0xFF;
    while(1) {
        for (uint8_t i = 0; i < 10; i++) {
            PORTD = digits[i];
            _delay_ms(10);
        }
    }
}
```

```
int digits[10] = { 0x3f, 0x06, 0x5b, 0x4f,
                  i {
                      DDAD = 0xFF;
                      while(1) {
                          for (int i = 0; i < 10; i++) {
                              PORTD = digits[i];
                              _delay_ms(10);
                          }
                      }
                  }
```

Q009 Rev

NEC

Toggle LCD

```
#include <avr/io.h>
#include <util/delay.h>
```

```
int main (void) {
    DDRD = 0xFF;
    while(1) {
        PORTD = ~PORTD;
        delay_ms(1000);
    }
}
```

```
#include <avr/io.h>
#include <util/delay.h>
#define switch PC0
int main (void) {
    DDRD = 0xFF;
```

```
    DDRC &= ~ (1 << switch)
```

```
    PORTD = (1 << switch)
```

```
    while(1) {
```

```
        if (PINC = 0x01) {
```

```
            PORTD = 0xFF;
```

```
        } else
```

```
            PORTD &= 0x00;
```

```
    }
}
```


4 bit LCD

```
# include <avr/io.h>
# define F_CPU 16000000
# include <util/delay.h>
# define LCD PORTD
# define EN 7
# define RW 6
# define RS 5
```

```
void lcd_cmd (unsigned char cmd) {
```

```
    PORTC &= ~ (1 << RS) ;
```

```
    PORTC &= ~ (1 << RW) ;
```

```
    LCD = cmd ;
```

```
    PORTC |= (1 << EN) ;
```

```
    _delay_ms (10) ;
```

```
    PORTC &= ~ (1 << EN) ;
```

```
}
```

```
void lcd_data (unsigned char data) {
```

```
    PORTC |= (1 << RS) ;
```

```
    PORTC &= ~ (1 << RW) ;
```

```
    LCD = data ;
```

```
    PORTC |= (1 << EN) ;
```

```
    _delay_ms (10) ;
```

```
    PORTC &= ~ (1 << EN) ;
```

```
}
```

00 -

```

void lcd_init ( ) {
    DDRD = 0xFF;
    DDRC = 0xFF;
    PORTC = ~ (1 << EN);
    lcdcmd (0x32);
    lcdcmd (0x28);
    lcdcmd (0x0e);
    lcdcmd (0x01);
    delay_ms (10);
}

```

```

void lcd_print (char * str) {
    unsigned char i = 0;

    while (str[i] != 0) {
        lcddata (str[i]);
        i++;
    }
}

```

```

int main (void) {
    lcd_init ();
    lcd_print (" Hello World");
    while (1);
    return 0;
}

```


S1, s2in, s3, ssum

to detect 1011

```
library ieee;  
use ieee.std_logic_1164.all;  
entity detect-1011 is  
port
```

```
(  
    clk, reset, seq_in : in std_logic;  
    detect : out std_logic);  
end detect-1011;
```

```
architecture arch_detect-1011 of detect-1011 is  
    type moore_fsm is (init, a, b, c, d);  
    signal c-st, n-st : moore_fsm;
```

```
begin  
    process (clk, reset)  
    begin  
        if (reset = '1') then  
            c-st <= init;  
        elsif (rising_edge(clk)) then  
            c-st <= n-st;  
        end if;  
    end process;
```

process (C-st, B & B-in)
begin

case (C-st) is
when init =>
detect L = '0' ;

delay of 10ms (imp)

include <avr/io.h>

void Todelay();

int main (void) {

DDRB = 0xFF;

while(1) {

PORTB = 0x55;

while(1) { Todelay();

PORTB = 0xAA;

Todelay();

}

void Todelay () {

TCNT = 0x63;

TCCR0 = 0x05;

while ((TIFR & 0x01) == 0) {

TCCR0 = 0;

TIFR = 0x01;

}

$$f = \frac{16\text{MHz}}{1024} = 15625\text{Hz}$$

$$T = \frac{1}{15625} = 0.000064\text{s}$$

$$\text{count} = \frac{1}{0.000064}$$

$$f_{\text{clk}} = 16\text{MHz}$$

$$= \frac{1000000}{16\text{MHz}}$$

$$= \frac{1000000}{16\text{MHz}}$$

$$f_{T0} = 15625\text{Hz}$$

$$\text{Count} = \frac{1}{f}$$

$$= \frac{1}{15625}$$

$$T = \frac{1}{f}$$

$$=$$

ES

detect 101

init = 0
a = 1
b = 10
c = 101

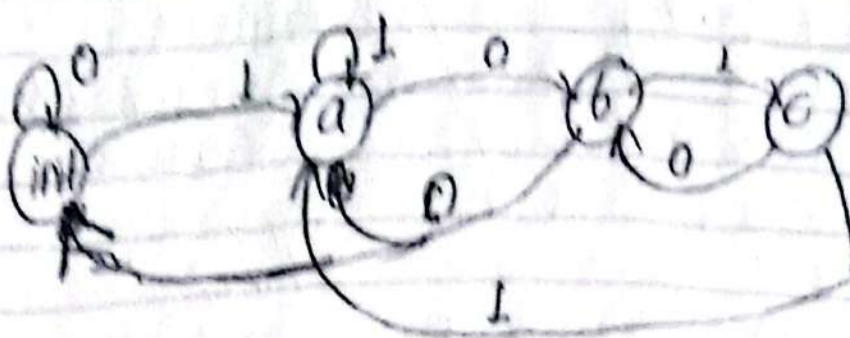
101

1010

1011

10101

100



library ieee;

use ieee.std_logic_1164.all;

entity detect_101 is

port (clk, reset, seq_in : in std_logic;
 det : out std_logic);

end detect_101;

architecture arch_detect_101 of detect_101 is

type moore_fsm is (init, a, b, c);

signal c_st, n_st : moore_fsm;

begin

process (clk, reset) begin

if (reset = '1') then

c_st <= init;

elsif (rising_edge(clk)) then

c_st <= n_st;

end if;

end process;

process (c_st, seq_in)

begin case c_st is

when init =>

det <= '0';

```

if (seq-in = '1') then
    n-st ← a;
else
    n-st ← init;
end if;

when b =>
    det ← '0';
    if (seq-in = '1') then
        n-st ← a;
    else
        n-st ← b;
    end if;

when c =>
    det ← 'q';
    if (seq-in = 'q') then
        n-st ← a;
    else
        n-st ← b;
    end if;
end case;
end process;
end arch-detect-101;

```


test bench

```
library ieee;
use ieee.std_logic-1164.all;
entity detect-101-tb is
end detect-101-tb;
architecture arch-detect-101-tb of detect-101-tb is
    component detect-101
        port (
            clk, reset, seq-in : in std_logic;
            dect : out std_logic);
        end component;
    signal clk : = '0';
    signal reset : = '0';
    signal seq-in : = '0';
    signal dect;
```

```
u1: detect-101 port map (
    clk => clk;
    reset => reset;
    seq-in => seq-in;
    dect => dect);
```

```
clk_process : process
begin
```

```
    clk <= '0';
    wait for 5 ns;
    clk <= '1';
    wait for 5 ns;
end process;
```

detect-process: process
begin

seq-in $\leftarrow$ '0';

reset $\leftarrow$ '1';

wait for 10ns;

reset $\leftarrow$ '0';

wait for 10ns;

seq-in $\leftarrow$ '1';

wait for 10ns;

seq-in $\leftarrow$ '0';

wait for 10ns;

seq-in $\leftarrow$ '1';

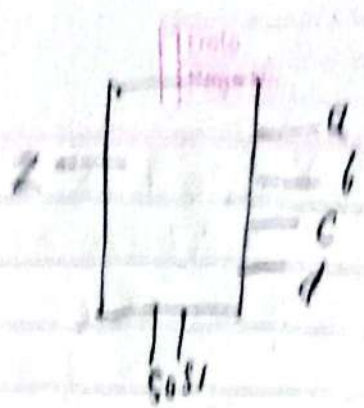
wait for 10ns;

wait;

end process;

end arch-detect-not-1b;

DEMUX 1:4



```
library ieee;
use ieee.std_logic_1164.all;
entity demux_1x4 is
    port (
        Z, S0, S1: in std_logic;
        a, b, c, d: out std_logic);
end demux_1x4;
```

architecture arch_demux_1x4 of demux_1x4 is
begin

```
    process (Z, S0, S1) is
    begin
        a <= '0';
        b <= '0';
        c <= '0';
        d <= '0';
        if (S0 = '0' and S1 = '0') then
            a <= Z;
        elsif (S0 = '0' and S1 = '1') then
            b <= Z;
        elsif (S0 = '1' and S1 = '0') then
            c <= Z;
        elsif (S0 = '1' and S1 = '1') then
            d <= Z;
        end if;
    end process;
end arch_demux_1x4;
```

Decoder 2x4:

library ieee;

use ieee.std_logic_1164.all;

entity decoder_2x4 is

port (

inputs : in std_logic_vector(1 downto 0);

en : in std_logic;

outputs : out std_logic_vector(3 downto 0);

architecture arch_decoder_2x4 of decoder_2x4 is
begin

process (inputs, en)

begin

if (en = '0') then

outputs <= '0000';

else

case inputs is

when '00' =>

outputs <= '0001';

when '01' =>

outputs <= '0010';

when '10' =>

outputs <= '0100';

when '11' =>

outputs <= '1000';

when others =>

Date _____
Page No. _____

```

outputs <= '0000';
end case;
end if;
end process;
end arch_decoder_2x4;

```

Encoder 4x2 :

```

library ieee;
use ieee.std_logic_1164.all;
entity encoder_4x2 is
    port (
        inputs : in std_logic_vector (3 downto 0);
        output : out std_logic_vector (1 downto 0);
        v : out std_logic);
end encoder_4x2;

```

architecture arch_encoder_4x2 of encoder_4x2 is
begin

<pre> v <= '1'; if (inputs = '0000') then output <= '00'; elsif (inputs = '0010') then output <= '01'; elsif (inputs = '0100') then output <= '10'; elsif (inputs = '1000') then output <= '11'; </pre>	<pre> else output <= '00'; v <= '0'; end if; end process; end arch_encoder_4x2; </pre>
---	--

Octal to binary converter

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
entity octalbinary is
```

```
port (
```

```
  oct_in input: in std_logic_vector (7 downto 0);
```

```
  b_out output: out std_logic_vector (2 downto 0)  
);
```

```
architecture arch_8x2 of octalbinary is  
begin
```

```
  process (inputs)
```

```
begin
```

```
  if (oct_in = '00000000') then  
    b_out <= '000';
```

```
  elsif (oct_in = '00000001') then  
    b_out <= '001';
```

```
  elsif (oct_in = '00000010') then  
    b_out <= '010';
```

```
  elsif (oct_in = '00000100') then  
    b_out <= '011';
```

```
  elsif (oct_in = '00010000') then  
    b_out <= '100';
```

```
  elsif (oct_in = '00100000') then  
    b_out <= '101';
```


elseif (out-in = '01000000') then
 b-out <= '110';
 elseif (out-in = '1000000') then
 b-out <= '111';

else
 b-out <= '000';
 end if;
 end process;
 end arch-8x2;

half subtractor = $\textcircled{1}$ = $A \oplus B$ $A \text{ XOR } B$

bout = $\overline{A} \cdot B$
 NOT(A) and B

BIT

BOOLEAN

INTEGER

- Scalar type
- 'Composit type
- Access types

Test Bench

It is an program written in VHDL that surrounds your actual hardware design (in like your main design, it does not get converted into hardware gates). Instead, it acts as a 'virtual tester'.

- generates input signal
- applies them to design under test.

VHDL Code for AND Gate.

```
library ieee;  
use ieee.std_logic-1164.all;  
entity and_gate is  
  port  
    a, b: in std_logic;  
    c: out std_logic;  
end and_gate;
```

```
architecture arch_and of and_gate is  
  begin  
    c <= a and b;
```


end arch-and;

The standard Procedure for Building Test Bench:

- Empty Entity Declaration
- Component Declaration
- Signal Declaration
- DUT (Design Under Test) instantiation
- clock Generation (if design is sequential)
- stimulates process

Test bench for AND gate

```
library ieee;  
use ieee.std-logic-1164.all;  
entity and-gate-tb is  
end and-gate-tb;
```

```
architecture arc-and-tb of and-gate-tb is  
  component and-gate  
    port (  
      a, b: in std-logic;  
      c: out std-logic);
```

```
end component;
```

```
signal sig-a: std-logic := '0';  
signal sig-b: std-logic := '0';  
signal sig-c: std-logic;
```

```
begin
```

w : and-gate port map (

a => sig-a;
b => sig-b;
c => sig-c);

atb: process
begin

sig-a <= '0';
sig-b <= '0';
wait for 100ps;

sig-a <= '0';
sig-b <= '1';
wait for 100ps;

sig-a <= '1';
sig-b <= '0';
wait for 100ps;

sig-a <= '1';
sig-b <= '1';
wait for 100ps;

wait;

end process;

end architecture arch-and-tb;

Test Bench for Half Adder

$$\text{sum} = A \oplus B$$

$$\text{carry} = A \text{ and } B$$

library ieee;

use ieee.std_logic_1164.all;

entity halfadder_tb is

end halfadder_tb;

architecture arc_half_tb of halfadder_tb is
component halfadder

port (

a, b : in std_logic;

s, c : out std_logic);

end component;

signal sig_a : std_logic := '0';

signal sig_b : std_logic := '0';

signal sig_s : std_logic;

signal sig_c : std_logic;

begin

u1: halfadder port map (

a => sig_a,

b => sig_b,

s => sig_s,

c => sig_c);

h1b: process // symbol

sig_a <= '0';

sig_b <= '0';

wait for 100ps;

```
sig_a <= '0';  
sig_b <= '1';  
wait for 100ps;
```

```
sig_a <= '1';  
sig_b <= '0';  
wait for 100ps;
```

```
sig_a <= '1';  
sig_b <= '1';  
wait for 100ps;  
wait;  
end process;  
end architecture arch_half_tb;
```

Q. Write test bench of 2x4 decoder and 4x2 encoder.

Soln: 2x4 decoder:

```
library ieee;  
use std_logic_1164.all;  
entity decoder_2x4 is  
port (en: in std_logic;  
      din: in std_logic_vector (1 downto 0);  
      dout: out std_logic_vector (3 downto 0));  
end decoder_2x4;
```

```
architecture arch_decoder_2x4 of decoder_2x4 is  
begin
```



```

dout(0) <= en and (not din(0))and(not din(0)) ;
dout(1) <= en and (not din(1)) and not din(0);
dout(2) <= en and din(1) and (not din(0));
dout(3) <= en and din(1) and din(0) ;
end arch-decoder-2x4 ;

```

```

library ieee;
use std_logic_1164.all;

```

```

entity decoder2x4 is
end decoder2x4;

```

```

architecture arch-decoder-2x4 of decoder2x4 is
component decoder2x4

```

```

port (
    en: in std_logic;
    din: in std_logic_vector (1 downto 0);
    dout: out std_logic_vector (3 downto 0)
);

```

```

end decoder2x4 component ;

```

```

signal sig-en: std_logic := '0';
signal sig-din: std_logic_vector (1 downto 0) := '0';
signal sig-dout: std_logic_vector (3 downto 0);

```

```

begin

```

```

    process

```

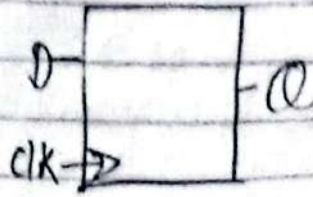
```

        sig-din <= '0';
    end process

```


VHDL Programs

1. VHDL Code for D-flipflop (Rising edge)



library ieee;

use std_logic_1164.all;

entity risingedge_dflipflop is

port (

clk : in std_logic;

q : out std_logic);

end risingedge_dflipflop;

(behavioral)
model) architecture arch_risingedge_dflipflop of risingedge_dflipflop is

begin

process (clk) -- predefined function

begin

if (rising_edge(clk)) then

q <= d;

end if;

end process;

end arch_risingedge_dflipflop;

(behavior = test bench)

Date _____
Page _____

VHDL Code for Testbench of D. flipflop.

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
entity tb_risingedge_dfflop is
```

```
end tb_risingedge_dfflop;
```

```
architecture arch_tb_risingedge_dfflop of  
    tb_risingedge_dfflop is
```

```
component risingedge_dfflop
```

```
port (
```

```
    d, clk : in std_logic;
```

```
    q : out std_logic);
```

```
end component;
```

```
signal sig_clk : std_logic := '0';
```

```
signal sig_d : std_logic := '0';
```

```
signal sig_q : std_logic;
```

```
constant CLK_PERIOD : time := 10ns;
```

```
begin
```

```
    u_dff : risingedge_dfflop port map (
```

```
        clk => sig_clk;
```

```
        d => sig_d;
```

```
        q => sig_q
```

```
    );
```

```

clk-process: process
begin
    clk_tb <= '0';
    wait for CLK_PERIOD/2;
    clk_tb <= '1';
    wait for CLK_PERIOD/2;
end process;

```

```

stim-process: process
begin
    wait for 20ns;

    sig-d clk <= '1';
    wait for 20ns;

    sig-d <= '0';
    wait for 15ns;

```

```

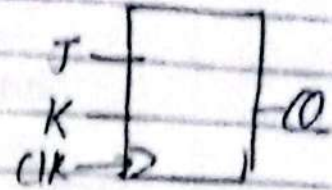
wait;
end process;
end arch_tb_risingedge_dfflop;

```

(instead risingedge) #
 falling state()

VHDL code for JK flipflop (rising edge)

```
ieee library ieee;
use std_logic_1164.all;
```



entity JK-flipflop is

port (

clk, j, k : in std_logic;

q : out std_logic

);

end JK-flipflop;

architecture arch-JK-flipflop of JK-flipflop is

begin

signal q-s : std_logic;

begin

process (clk)

begin

if (rising-edge (clk)) then

if (j = '0' and k = '0') then

q-s ≤ q-s;

elsif (j = '1' and k = '0') then

q-s ≤ '1';

elsif (j = '0' and k = '1') then

q-s ≤ '0';

elsif (j = '1' and k = '1') then

q-s ≤ not q-s;

end if;

~~end if;~~ end if;

end process;

q <= q-s;

end arch-JK-Flipflop;

(q-bar <= q-s)

(falling)

same

Testbench

library ieee;

use std_logic_1164.all;

entity JK-Flipflop-tb is

end JK-Flipflop-tb;

architecture arch-JK-Flipflop-tb of JK-Flipflop-tb is

component JK-Flipflop

port (

clk, j, k : in std_logic;

q : out std_logic);

end component;

signal sig_clk : std_logic := '0';

signal sig_j : std_logic := '0';

signal sig_k : std_logic := '0';

signal sig_q : std_logic;

~~signal q-s : std_logic;~~

begin

u1: JK-Flipflop port map (

clk $\rightarrow$ sig-clk;
 j $\rightarrow$ sig-j;
 k $\rightarrow$ sig-k;
 q $\rightarrow$ sig-q;
);

clb: process
 sig-clk $\leftarrow$ '0';
 wait for 10ns;
 sig-clk $\leftarrow$ '1';
 wait for 20ns;
 end process;

jkb: process
 begin
 sig-j $\leftarrow$ '0';
 sig-k $\leftarrow$ '0';
 wait for 10ns;

sig-j $\leftarrow$ '0';
 sig-k $\leftarrow$ '1';
 wait for 10ns;

sig-j $\leftarrow$ '1';
 sig-k $\leftarrow$ '0';
 wait for 10ns;

sig-j $\leftarrow$ '1';
 sig-k $\leftarrow$ '1';
 wait for 10ns;
 wait;

2024 Fall

VHDL code for MUX 4:1

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
entity mux4to1 is  
  port (  
    s0, s1, a, b, c, d : in std_logic;  
    z : out std_logic  
  );
```

architecture arch-mux-4to1 of mux4to1 is
 begin

process (a, b, c, d, s0, s1) is

begin

if (s0 = '0' and s1 = '0') then

z <= a;

elsif (s0 = '0' and s1 = '1') then

z <= b;

elsif (s0 = '1' and s1 = '0') then

z <= c;

elsif (s0 = '1' and s1 = '1') then

z <= d;

end if;

end process;

end arch-mux-4to1;

~~end if~~

test bench

```
library ieee;  
use std_logic_1164.all;  
entity mux_4to1_tb is  
end mux_4to1_tb;
```

```
architecture arch_mux_4to1_tb of mux_4to1_tb  
component mux_4to1  
port (  
    s0, s1, a, b, c, d : in std_logic;  
    z : out std_logic);  
end component;
```

```
signal sig_s0 : std_logic := '0';  
signal sig_s1 : std_logic := '0';  
signal sig_a : std_logic := '0';  
signal sig_b : std_logic := '0';  
signal sig_c : std_logic := '0';  
signal sig_d : std_logic := '0';  
signal sig_z : std_logic;
```

```
begin  
u1 : mux_4to1 port map (  
    s0 => sig_s0,  
    s1 => sig_s1,  
    a => sig_a,  
    b => sig_b,  
    c => sig_c,  
    d => sig_d,  
    z => sig_z  
);
```

(same a and sig-a)
easy

stim-proc : process

begin

wait for 10ns ;

sig-a $\leftarrow$ '1' ;

sig-b $\leftarrow$ '0' ;

sig-c $\leftarrow$ '1' ;

sig-d $\leftarrow$ '0' ;

so $\leftarrow$ '0' ; si $\leftarrow$ '0' ;
wait for 10ns ;

so $\leftarrow$ '0' ; si $\leftarrow$ '1' ;
wait for 10ns ;

so $\leftarrow$ '1' ; si $\leftarrow$ '0' ;
wait for 10ns ;

so $\leftarrow$ '1' ; si $\leftarrow$ '1' ;
wait for 10ns ;

wait ;

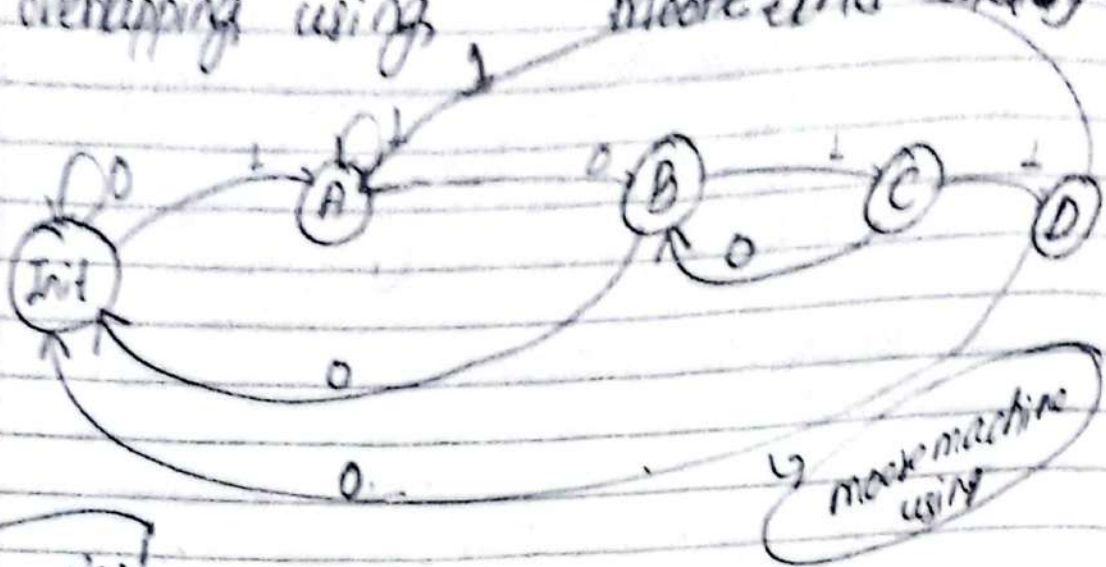
end process ;

end arch-mux-4to1-tb ;

Moore

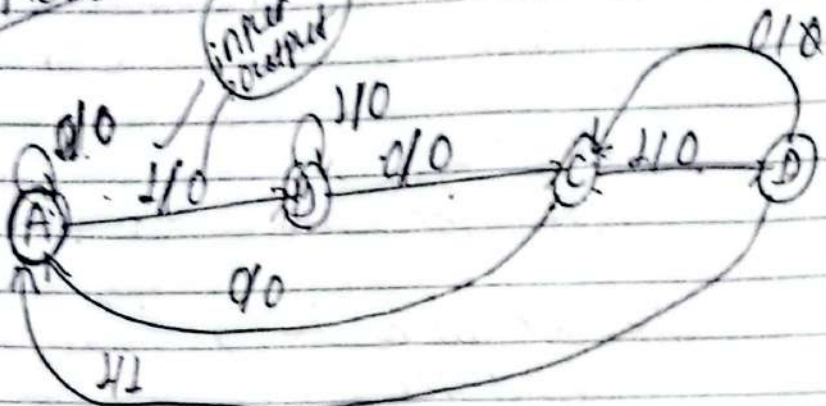
$A = 1$
 $B = 10$
 $C = 101$
 $D = 1011$

Q. Design state diagram for a sequence detector that detects 1011 without overlapping using Moore and delay.



without overlap
mean final
state initial state

delay machine
input output



1011D

D state is 1
error, 1011111

1011

1010

1010
 detect.out = '1';
 detect.out = '0';

1010

VHDL code for 1011 (M.P.)

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
entity detect_1011 is  
  port (  
    clk, reset, seq_in : in std_logic;  
    detect_out : out std_logic  
  );
```

```
end detect_1011;
```

```
architecture arch_detect_1011 of detect_1011 is
```

```
  type mela_fsm is ( a, b, c, d );  
  signal c_st, n_st : mela_fsm;
```

```
begin  
  process (clk, reset)  
  begin  
    if (reset = '1') then  
      c_st <= 0 a;  
    elsif (rising_edge(clk)) then  
      c_st <= n_st;
```

```
    end if;  
  end process;  
  process (c_st, seq_in)  
  begin  
    case (c_st) is
```


when a =?
detect-out = '0' ;

if (seq-in = '1')
n-st <= b;
elseif
n-st <= a;
end if ;

when b =?
detect-out = '0' ;
if (seq-in = '1')
n-st <= b;
elseif
n-st <= a;
end if ;

when c =?
detect-out = '0' ;
if (seq-in = '1')
n-st <= d;
elseif
n-st <= a;
end if ;

when d =?
if (seq-in = '1')
detect-out = '1' ; n-st <= a;
elseif
detect-out = '0' ;
n-st <= c;
end if ;

Test

en

or

com

Sig → bgho

Post

Pro

Date _____
Page _____

```
end case;  
end process;  
end arch-detect-1001;
```

test bench

```
library ieee;  
use ieee.std_logic-1164.all;  
entity detect-1001-tb is  
end detect-1001-tb;
```

```
architecture arch-detect-1001-tb of detect-1001-tb  
is  
component detect-1001  
port (  
    clk, reset, seq-in: std_logic;  
    detect-out: out std_logic);  
end component;
```

```
signal sig-clk; = '0';  
signal sig-reset; = '0';  
signal sig-seq-in; = '0';  
signal sig-detect-out;
```

```
begin
```

```
u: detect-1001 port map (  
    clk => sig-clk,  
    reset => sig-reset,  
    seq-in => sig-seq-in,  
    detect-out => sig-detect-out  
);
```


clk process: process

begin

sig-clk <= '0' ;

wait for 5ns ;

sig-clk <= '1' ;

wait for 5ns ;

end process ;

detect-out-process: process

begin

sig-seq-in <= '0' ;

sig-reset <= '1' ;

wait for 10ns ;

sig-reset <= '0' ;

wait for 10ns ;

sig-seq-in <= '1' ;

wait for 10ns ;

sig-seq-in <= '0' ;

wait for 10ns ;

sig-seq-in <= '1' ;

wait for 10ns ;

sig-seq-in <= '0' ;

wait for 10ns ;

wait ;

end process ;

end arch-detect-10k-tb ;



Eg:

```
#include <avr/io.h>
#include <util/delay.h>
```

20
input = pull up register activate

```
int main (void) {
```

```
    DDRA = 0xFF;
```

```
    DDRB = 0x00;
```

```
    PORTB = 0xFF; [Pull up register]
```

in DDR
00 = input
01 = output

```
    unsigned char counter = 0;
```

```
    while(1) {
```

```
        if (PINB & (1 << PB0)) {
```

```
            PORTA = 0x00; // turn off all LED.
```

```
        } else {
```

```
            PORTA = 0x01; // PA0 in on
```

```
        for (counter = 0; counter < 8; counter++) {
```

```
            PORTA = (1 << counter);
```

```
            - delay_ms(200);
```

turn off after delay.

```
            PORTA = 0x00;
```

```
            - delay_ms(500);
```

```
        }
```

```
        - delay_ms(10);
```

```
    }
```

```
}
```

Q. WAP for LED chaser using loop and delay systems.

WAP

PORT - output
PIN - input

Q2. WAP for AVR microcontroller that takes input from port A1. (port A pin 1) control the LED connected at port C 7 (PC7).

```
Soln: #include <avr/io.h>
#include <util/delay.h>
int main(void) {
    DDRA = 0x01;
    PORTC = 0x07;
    // configure port A pin 1 as input
    DDRA &= ~(1 << PA1);

    // enable internal pull-up register
    PORTA |= (1 << PA1);

    // configure PC7 as output:
    DDRC |= (1 << PC7);

    // main loop
    while(1)
    {
        if (! (PINA & (1 << PA1))) {
            PORTC |= (1 << PC7); // turn LED on
        }
        else {
            // turn LED off
            PORTC &= ~(1 << PC7);
        }
    }
}
```

1st term, dec 9

6b) WAP a VHDL program for full adder using structural modelling.

soln: library ieee;
use ieee.std_logic-1164.all;
entity ~~half-adder~~ half-adder is
port (
h-a, h-b: in std_logic;
h-c, h-s: out std_logic);
end half-adder;

architecture arch-half-adder of half-adder is
begin
h-s $\leq$ h-a xor h-b;
h-c $\leq$ h-a and h-b;
end arch-half-adder;

library ieee;
use ieee.std_logic-1164.all;
entity full-adder is
port (
f-a, f-b, f-cin: in std_logic;
f-sum, f-cout: out std_logic);
end full-adder;

architecture arch-full-adder of full-adder is
component half-adder
port (
h-a, h-b: in std_logic;
h-s, h-c: out std_logic);


```

end component;
signal s1, s2, s3: std_logic;
begin
  u1: half_adder port map (f.a, f.b, s1, s2);
  u2: half_adder port map (s1, f.cin, s3, f.sum);
  f.cout <= s1 or s3;
end arch_full_adder;

```

~~f.a, f.b, s1, s2~~
~~s1, f.cin, s3, f.sum~~
~~f.cout <= s1 or~~

8. VHDL code for half adder using structural modelling.

```

S1: library ieee;
    use ieee.std_logic_1164.all;
    entity and_gate is
    port (
        a, b: in std_logic;
        c: out std_logic;
    end and_gate;

```

```

architecture arch_andgate of and_gate is
begin
    c <= a and b;
end arch_andgate;

```

```

library ieee;
    use ieee.std_logic_1164.all;
    entity xor_gate is
    port (
        a, b: in std_logic;
        c: out std_logic;
    end xor_gate;

```

```

architecture arch_xorgate of xor_gate is
begin
    c <= a xor b;
end arch_xorgate;

```

```

library ieee;
    use ieee.std_logic_1164.all;
    entity half_adder is
    port (

```

```

        a, b: in std_logic;
        sum, carry: out std_logic;
    end half_adder;

```

```

architecture arch_half_adder of half_adder is
    component and_gate
    port (

```

```

        a, b: in std_logic;
        c: out std_logic;
    end component;

```

```

    component xor_gate
    port (

```

```

        a, b: in std_logic;
        c: out std_logic;
    end component;

```

```

begin
    u0: and_gate port map (
        a => a,
        b => b,
        c => carry;

```

```

    u1: xor_gate port map (
        a => a,
        b => b,
        c => sum;
    end arch_half_adder;

```


library ieee;

use ieee.std_logic_1164.all;

entity half-adder is

port (

a, b: in std_logic;

sum, carry: out std_logic);

end half-adder;

architecture arch_half_adder of half-adder is

component and_gate

port (

a, b: in std_logic;

c: out std_logic);

end component;

component xor_gate

port (

a, b: in std_logic;

c: out std_logic);

end component;

begin

u0: and_gate port map (

a => a;

b => b;

c => carry);

u1: xor_gate port map (

a => a;

b => b;

c => sum);

end arch_half_adder;

preuniversity21

1. b) Write a C program to configure PORT A and PORT B as output port and port C and port D as input port. Also receive data from C and D and send them to A and B respectively.

Solⁿ: # include <avr/io.h>

int main (void) {

DDRA = 0xFF; (A as output)
 DDRB = 0xFF;

DDRC = 0x00;
 DDRD = 0x00;

// enable pullup register

PORTC = 0xFF; [enable to all pin]
 PORTD = 0xFF; 11111111

while(1) {

PORTA = PINC;
 PORTB = PIND;

}
 return 0;

}


```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h>
#include <stdio.h>
#define BAUD_PRESCALE (((F_CPU/(USART_BAUDRATE * 16UL))) - 1)
```

```
void UART_init (long USART_BAUDRATE)
{
    UCSRB = (1 << RXEN) | (1 << TXEN);
    UCSRC = (1 << URSEL) | (1 << UCSZ0) | (1 << UCSZ1);
    UBRRH = (BAUD_PRESCALE > 78); UBRRL = BAUD_PRESCALE;
}
```

```
unsigned char UART_RxChar()
{
    while ((UCSRA & (1 << RXC)) == 0);
    return UDR;
}
```

```
void UART_TxChar (char ch)
{
    while (!(UCSRA & (1 << UDRE)));
    UDR = ch;
}
```

```
void UART_SendString (char * str)
{
    unsigned char j = 0;
    while (str[j] != 0)
    {
        UART_TxChar (str[j]);
        j++;
    }
}
```

URSEL = 1
if 0 UBRR

Date _____
Page _____

(protos)

{
}

int main ()

{

char c;

UART_init (9600);

UART_SendString ("Init Echo Test");

while (1)

{

c = UART_RxChar ();

UART_TxChar (c);

} receive transmit

{
}

// UBRR = calculate ;
UBRRH = 00000000;

UBRR

(UBRRH) 8H
(UBRR) 8L
7H

WAP for ATmega32 to create delay
of 1s using timer0.

using interrupt

```
#include <avr/io.h>
#include <avr/interrupt.h>

/* timer0 overflow interrupt */
ISR (TIMER0_OVF_vect)
{
    PORTD = ~PORTD; /* toggle PORTD
    TCNT0 = 0x01;
}

int main(void)
{
    DDRD = 0xff; /* port D as output
    PORTD = 0x05;
    sei();
    TIMSK |= (1 << TOIE0); /* enable timer0
                             overflow interr
    TCNT0 = 0x01; /* load overflow
    TCCR0 = (1 << CS02) | (1 << CS00);
    while(1);
}
```

imp
- LCD question

LM85 (Assembly)

```
include "m82def.inc"  
org 0x0000
```

```
LDI R16, HIGH(RAMEND) // initialization  
OUT SPH, R16  
LDI R16, LOW(RAMEND)  
OUT SPL, R16
```

```
SER R16 // PORTB = 0  
OUT DDRB, R16
```

```
// ADC setup  
LDI R16, (1 << REFS0)  
OUT ADMUX, R16
```

```
LDI R16, (1 << ADEN) | (1 << ADPS2) | (1 << ADPS2)  
OUT ADCSRA, R16
```

MAIN:

```
SBI ADCSRA, ADSC
```

WAIT:

```
SBI ADCSRA, ADSC  
RJMP WAIT
```

// read result

```
IN R17, ADCL
```

```
IN R18, ADCH
```

// convert to Celsius

```
LSR R18
```

```
ROR R17
```

```
OUT PORTB, R17  
RJMP MAIN
```


code

(Normal mode)

delay 10ms using Timer 0.

$$\text{count} = \frac{10 \times 1000}{1024/16} = 156$$

CS₀ CS₁ CS₂ CS₃ CS₄ CS₅ CS₆ CS₇ CS₈ CS₉

$$\text{TCNT} = 255 - 156 = 99 = 0x63$$

```
#define F_CPU 16000000
#include <avr/io.h>
void Todelay();
int main(void) {
    DDRB = 0xFF;
    while(1) {
        PORTB = 0x55;
        Todelay();
        PORTB = 0xAA;
        Todelay();
    }
}
```

```
void Todelay() {
    TCNT0 = 0x63;
    TCCR0 = (1 << CS0) | (1 << CS00);
    while((TIFR & 0x01) == 0);
    TCCR0 = 0;
    TIFR = 0x01;
}
```

2024 Fall

square wave using timer0 & timer1

2b)

```
#include <avr/io.h>
```

```
void timer0() {
```

```
    DDRB1 = (1 << PB3);
```

```
    TCCR01 = (1 << WGM01) | (1 << CM00) | (1 << CS01);
```

```
    OCR0 = 64;
```

```
}
```

```
void timer1() {
```

```
    DDRD1 = (1 << PD5);
```

```
    TCCR1A1 = (1 << COM1A0);
```

```
    TCCR1B1 = (1 << WGM12) | (1 << CS10);
```

```
    OCCRA1A = 499;
```

```
}
```

```
int main(void) {
```

```
    timer0();
```

```
    timer1();
```

```
    while(1);
```

```
}
```


2024 Fall

2a) toggle LEDs

```
# include <avr/io.h>
```

```
# include <util/delay.h>
```

```
int main (void) {
```

```
    DDRA = 0xFF;
```

```
    while (1) {
```

```
        PORTA = ~PORTA;
```

```
        delay_ms(200);
```

```
    }
```

```
}
```

2a) LED turn on/off

```
# include <avr/io.h>
```

```
# include <util/delay.h>
```

```
# define switchpin PC0
```

```
int main (void) {
```

```
    DDRA = 0xFF;
```

```
    DDRC &= ~(1 << switchpin);
```

```
    PORTC |= (1 << switchpin); // pulloff
```

PC0=1

```
while (1) {
```

```
    if (PINC = 0x01) {
```

```
        PORTD |= 0xFF;
```

```
    }
```

```
    else {
```

```
        PORTD &= 0x00;
```

```
    }
```

```
}
```

```
}
```

PROBLEM Square wave of 50% duty cycle PORTB, (timer 0 to timedelay)

26) #include <avr/io.h> int main(void) {
 DDRB |= (1 << PB5);

TCCR0A |= (1 << WGM01) | ~~4~~ ~~CTE mode~~
 (1 << CS02) | (1 << CS00);

OCR0 = 156;

while (1) {
 while ((TIFR & 0x02) == 0);

PORTB ^= (1 << PB5);

TIFR = 0x02;

}

}

→ Receive byte of data serially and out them on Port A baudrate 9600, 8 bit data and 1 stop bit

#define F_CPU 8000000

#include <avr/io.h>

#define BAUD-PRESCALE ((F_CPU/(baud*16))-1)

void UART_init(long baud) {

UCSRB |= (1 << RXEN);

USRC |= (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0);

UBRRH = (BAUD-PRESCALE >> 8); UBRRL = BAUD-PRESCALE;

unsigned char Rx;

while ((USCRA & (1 << RXIF)) == 0);

return UDR;

}


```
int main (void) {  
    DDRA = 0xFF;
```

```
    while (1) {  
        PORTA = 0xFF;
```

```
    }  
}
```

yy

preboard ES

NAIST

Q.6) WAP to read data from PORTB and if the value is greater than 100 send it to PORTC, otherwise clear PORTC

```
#include <avr/io.h>
int main (void) {
    DDRA = 0x00;
    DDRC = 0xFF;

    if ( PINB > 100 ) {
        PORTC = PINB;
    } else {
        PORTC = 0x00;
    }

    return 0;
}
```

Roles of Membership function:

1. Quantifying ambiguity:

NEC

WAP to interface LED to blinks ON and OFF using time 0 in normal mode. generate 1s delay calculation.

```
#include <avr/io.h>
#define F_CPU 16000000
```

```
void delay() {
    for (int i = 0; i < 100; i++) {
```

```
        TCNT = 0x64;
        TCCR0 = (1 << CS02) | (1 << CS01);
        while ((TIFR & 0x01) == 0);
        TCCR = 0;
        TIFR = 0x01;
    }
}
```

```
int main() {
    DDRB = (1 << PB0);
    while(1) {
        PORTB = (1 << PB0);
        delay();
    }
    return 0;
}
```

$$f = 16 \text{ MHz}$$

$$T = \frac{1024}{16} = 64 \mu\text{s}$$

$$\text{count} = \frac{10 \times 1000}{64}$$

$$= 156$$

$$\text{TCNT} = 255 - 156 = 99$$

$$= 0x64$$

DDRB = 0xFF;

while(1) {

PORTB = 0x55;

delayms(1);

PORTB = 0xAA;

VHDL modelling style

1. Data Overflow model

Describe how data flows from input to output using concurrent signal assignments.

gate syntax:

```
entity and_gate is
    port (
        a, b : in std_logic;
        c : out std_logic);
end and_gate;
```

architecture arch_and of and_gate is

begin

```
c <= a and b;
end arch_and;
```

2. Behavioral Modelling

Describe how the circuit behaves using sequential statement (inside a process). High level abstraction uses if-else.

entity entity_name is

port (

a, b : in std_logic;

c : out std_logic); end

architecture arch_name of entity_name is

begin

```
process ( ) begin if ... else end process
code ... end arch-
```


3. Structure Model

Describe the circuit as a collection of interconnected components. It defines what components are used and how they are wired.

26) WAP to display "Hello World" on UART with default configuration.

Solⁿ:

```
#include <avr/io.h>
#define F_CPU 16000000
#include <avr/delay.h>

void uart_init(void) {
    UCSRB1 = (1 << TXEN);
    UCSRC1 = (1 << UCSZ1) | (1 << UCSZ0) |
              (1 << URSEL);
    UBRR1 = 0x33;
}

void uart_send(char ch) {
    while (!(UCSRA & (1 << UDRE)));
    UDR = ch;
}

int main(void) {
    char str[] = "Hello World";
    uart_init();
    while(1) {
```

```
for (int i = 0; str[i] != '\0'; i++)  
{  
    uart_send (str[i]);  
    delay_ms (1000);  
}  
return 0;  
}
```


COSMOS

WAP to generate square wave of 8 μ s on PB5 using timer 0 overflow interrupt while at same time data is being transferred from PORTC to PORTD. (8MHz)

solⁿ: #include <avr/io.h>
#include <avr/interrupt.h>

ISR (Timer0_OVF_vect) {

$$f = 8 \text{ MHz}$$

$$\text{Square wave} = 8 \mu\text{s}$$

$$\text{Half period} = 4 \mu\text{s}$$

$$T = \frac{1}{8 \text{ MHz}} = 0.125 \mu\text{s}$$

$$\text{count} = \frac{4}{0.125 \mu\text{s}}$$

$$T = 32$$

$$\text{TCNT} = 256 - 32 = 224$$

PORTB = (1 << PB5);

TCNT0 = 224; OX01;

int main (void) {

DDRB = (1 << PB5);

DDRC = 0x00;

DDRD = 0xFF; sei();

TCNT0 = 224;

TIMSK0 = (1 << TOIE0);

TCCR0 = (1 << CS00);

sei();

while (1) {

PORTD = PINC;

}

EEPROM code;

```
#include <avr/io.h>
#include <avr/eeprom.h>
```

```
uint8_t EEMEM var;
int main(void) {
```

```
    uint8_t value = 55;
    uint8_t readvalue;
```

```
    eeprom_write_byte(&var, value);
    readvalue = eeprom_read_byte(&var);
    while(1);
```

```
}
```


4) Watchdog Timer: reset microcontroller if software hangs or enters a infinite loop.

2b) Memory managed in embedded C.

Flash memory:

SRAM

EEPROM

Stack: LIFO

```
code: #include <avr/io.h>
      #include <pgmspace.h>
```

```
int gbl_var = 50;
int stackflag; }
```

```
const char message[] PROGMEM = "System online";
void check (int input) {
    int local_temp = 0;
    local_temp = input + gbl_var;
```

```
}
int main (void) {
    check(10);
    while(1);
}
```

example .

```
#include  
library ieee;  
use ieee.std_logic_1164.all;
```

```
package pac_clk is  
    type clk_type is ('0', '1');  
    function rising_edge (signal s: clk_type) return boolean;  
end pac_clk;
```

```
package body pac_clk is  
    function rising_edge (signal s: clk_type) return boolean is  
begin  
    if ((s'event) and (s = '1') & (s'last_value = '0'))  
    then  
        return true;  
    else  
        return false;  
    end if;  
end rising_edge;  
end package body pac_clk;
```